

Hosting, Payments, VIP Automation, Forums & Database Deployment Guide

Version 4.7 - Production migration handbook

For the included static HTML/CSS/JavaScript site and optional Node.js membership server

IMPORTANT: *The current browser-only member and forum system is a local preview. LocalStorage is controlled by each visitor and is not a secure production database. Automatic paid VIP access must be verified by a trusted server webhook and saved in a database. Never put Stripe secret keys, database passwords, or a Supabase service-role key in browser JavaScript.*

Contents

1. Choose the correct hosting architecture
2. Prepare the ZIP before deployment
3. Free static hosting: GitHub Pages, Netlify, Cloudflare Pages, Vercel
4. Paid/shared hosting: cPanel and managed Node hosting
5. Domains, DNS, HTTPS, and email
6. Connect Stripe Checkout and the VIP paywall
7. Automatic VIP badge and music access flow
8. MySQL setup for users, forums, replies, and purchases
9. Supabase alternative: Auth + Postgres + RLS
10. Moving the current localStorage system to production
11. Security, backups, testing, and launch checklist
12. Troubleshooting and official references

1. Choose the correct hosting architecture

The visual website can be hosted as a static site. Secure login, forums shared by all visitors, payment verification, VIP badges, and protected downloads require a backend and database. Use one of these patterns:

Pattern	Front end	Backend/database	Best use
Static preview	GitHub Pages / Netlify / Cloudflare Pages / Vercel	None; localStorage only	Portfolio/demo; not secure accounts or paid access
Recommended managed	Static host	Supabase Auth + Postgres + Edge Function, or hosted Node API	Fastest production path with real users/forums
Traditional shared hosting	cPanel public_html	MySQL + PHP API, or Node if host supports it	One hosting account and familiar control panel
Flexible production	Static host or Nginx	Node.js API + MySQL/Postgres on Render/Railway/VPS	Maximum control and Stripe webhook support

2. Prepare the ZIP before deployment

- Make a backup of the entire project and browser data from Admin Hub -> Performance & Storage.
- Extract the ZIP. The hosting root must contain index.html directly, not an extra parent folder.
- Do not upload .env files, database dumps, node_modules, or secret keys into public_html or a public Git repository.
- Set js/backend-config.js apiUrl to the HTTPS URL of the deployed membership API.
- Test index.html, music.html, members.html, forums.html, admin.html, images, CSS, navigation, and mobile layout before changing DNS.

3. Free static hosting

GitHub Pages

Create a repository; upload the site files so index.html is at the published root; open Settings -> Pages; choose the branch/folder; wait for the Pages URL; add a custom domain in Pages settings; enable Enforce HTTPS.

Backend note: Excellent source control and free static hosting. It cannot run the included Node server or MySQL. Host the API elsewhere.

Netlify

Sign in; create a project; either connect the Git repository or drag the extracted project folder into Netlify Drop; verify index.html is at the root; use Domain management for a custom domain.

Backend note: Easy manual deployment and serverless options. Put secrets in environment variables, never backend-config.js.

Cloudflare Pages

Workers & Pages -> Create application -> Pages; connect Git or use Direct Upload; for plain HTML use no build command and the site folder as output; configure the custom domain in Pages.

Backend note: Good global static delivery. Use Workers/Pages Functions or a separate API for payments and database work.

Vercel

Import the Git repository or deploy with the CLI; no framework is required for static files; verify the assigned .vercel.app URL; add the domain under Project Settings -> Domains.

Backend note: Good for static hosting and serverless functions. The included Express server may be adapted to functions or hosted separately.

4. Paid and non-free hosting

Shared hosting with cPanel

13. Open File Manager and enter public_html.

14. Remove the host placeholder page, then upload the ZIP or extracted files. If uploading a ZIP, extract it inside `public_html`.
15. Confirm `public_html/index.html` exists. Fix permissions only when necessary: files commonly 644 and directories 755.
16. Use MySQL Database Wizard to create a database, database user, and strong password; grant only required privileges.
17. Import `Documentation/database/mysql-schema.sql` with phpMyAdmin.
18. A basic shared plan may not run Node.js. Choose a plan with Setup Node.js App, use a PHP API instead, or host the included Node server on a separate service.
19. Enable the host SSL certificate and force HTTPS after DNS resolves.

Managed Node host or VPS

20. Upload the server folder to a private Git repository.
21. Set all values from `server/.env.example` in the host environment-variable panel.
22. Run `npm install` and `npm start`.
23. Expose port 8787 through the platform or reverse proxy it behind `https://api.your-domain.com`.
24. Set `ALLOWED_ORIGINS` and `PUBLIC_SITE_URL` to the exact HTTPS website origin.
25. On a VPS, use a process manager and Nginx/Caddy, firewall unused ports, enable automatic security updates, and configure daily database backups.

5. Domains, DNS, HTTPS, and email

Buy a domain from a registrar, then follow the hosting provider's exact DNS values. An apex domain usually uses A/AAAA or provider nameservers; a subdomain commonly uses CNAME. DNS changes can take time. Do not remove working mail MX records when changing web DNS. Enable HTTPS before activating login or checkout, and redirect all HTTP traffic to HTTPS.

6. Connect Stripe Checkout and the VIP payroll

26. Create a Stripe account and complete required business verification.
27. Create a VIP product and a recurring Price for monthly/yearly access, or a one-time Price for lifetime access. Copy the `price_...` ID.
28. Deploy `server/payment-membership-server.js` on a server that can receive public HTTPS POST requests.
29. Copy `server/.env.example` to a private `.env` file. Set `STRIPE_SECRET_KEY`, `STRIPE_VIP_PRICE_ID`, `PUBLIC_SITE_URL`, database values, and `STRIPE_VIP_MODE`.
30. In Stripe Workbench/Webhooks, add `https://api.your-domain.com/api/webhooks/stripe`. Subscribe at minimum to `checkout.session.completed`; for subscriptions also use `customer.subscription.updated` and `customer.subscription.deleted`.
31. Copy the endpoint signing secret (`whsec_...`) into `STRIPE_WEBHOOK_SECRET`.
32. In `js/backend-config.js`, set `apiBaseUrl` to the deployed API. Do not put secret keys there.
33. Test with Stripe test mode. Confirm a completed payment changes `member_profiles.vip_active`, then reload `music.html` and verify the VIP badge and vault access.
34. Only after tests pass, replace test keys and test Price ID with live values, then create the live webhook endpoint.

Do not unlock VIP from the success-page redirect alone. A user can visit that URL without paying. The signed Stripe webhook is the source of truth. Stripe recommends webhooks for reliable fulfillment.

7. How automatic VIP access works in this ZIP

Checkout request: music.html asks the backend to create a Checkout Session for the signed-in member.

Stripe payment: The customer pays on Stripe-hosted Checkout.

Verified webhook: Stripe sends a signed event to the backend; the server verifies the signature and records the event ID so retries are safe.

Database update: member_profiles.vip_active becomes true and stores Stripe customer/subscription identifiers.

Browser synchronization: After returning to music.html, vip-membership-v4.7.js requests the member status and updates the current local badge/access display.

Manual override: Admin Hub -> Members can still grant/remove paid access in local preview. In production, create an authenticated admin API that updates the database and writes an audit log.

Production identity requirement: the included server permits X-Demo-User-Id only when ALLOW_LOCAL_PREVIEW_IDENTITIES=true. That setting is for development. Before production, validate a real login token (for example Supabase Auth JWT) and derive the user ID on the server. Never trust a user ID sent by the browser.

8. MySQL setup for accounts and forums

Documentation/database/mysql-schema.sql creates users, member_profiles, forum categories, posts, replies, payment events, and purchase entitlements. Import it into a new database. The included Node server already reads member_profiles and writes verified VIP changes.

Recommended API routes

- POST /api/auth/register - hash passwords with Argon2id or bcrypt; never store plain passwords.
- POST /api/auth/login - issue a secure, HttpOnly, SameSite cookie or validated token.
- GET/PATCH /api/profile - owner access; admin fields protected server-side.
- GET/POST /api/forums/posts - pagination and validation.
- POST/DELETE /api/forums/posts/:id/replies - owner/admin authorization.
- POST /api/checkout/vip - authenticated users only.
- POST /api/webhooks/stripe - raw body, signature verification, idempotency.
- GET /api/membership/status - return the authenticated user's record, not an arbitrary user ID.

Database rules

- Use parameterized queries; the included mysql2 code does this.
- Create a database user for the application instead of using root.
- Add indexes for post dates, user IDs, category IDs, and Stripe identifiers.
- Use transactions for multi-table writes.
- Log admin role/VIP changes and webhook event IDs.
- Back up the database daily and test restoration.
- Store uploaded files in object storage; save only file metadata and protected URLs in MySQL.

9. Supabase alternative

Supabase combines Auth, Postgres, Storage, and Row Level Security. Run `Documentation/database/supabase-schema.sql`, configure Auth redirect URLs, and replace the local `members/forums` scripts with Supabase client calls. Use the anon key in the browser only with RLS enabled. Keep the service-role key exclusively in a trusted server or Edge Function.

- 35. Auth creates the user identity and returns a JWT.
- 36. `profiles` stores role and VIP status; members may read their own profile but must not update `vip_active`.
- 37. `forum_posts` and `forum_replies` use `auth.uid()` ownership policies.
- 38. A Stripe webhook Edge Function uses the service role to update VIP fields.
- 39. Supabase Storage policies protect private audio and gallery uploads; VIP downloads should use short-lived signed URLs.

10. Migrate current localStorage data

- 40. Export browser data from Admin Hub before migration.
- 41. Create real accounts in the new authentication system; do not copy existing plain-text demo passwords.
- 42. Write a one-time admin import script that maps users to database UUIDs.
- 43. Import forum posts and replies after sanitizing HTML and validating ownership.
- 44. Upload media to object storage and replace base64/local paths with URLs.
- 45. Keep the old localStorage keys read-only during a transition period, then remove the local write paths after production verification.
- 46. Do not merge browser-paidMember values into production VIP status. Reconcile paid access from Stripe records or an approved admin list.

11. Security, backups, and launch checklist

- ☐ HTTPS is enforced on the website and API.
- ☐ Secrets exist only in server environment variables.
- ☐ Real authentication replaces localStorage passwords and X-Demo-User-Id.
- ☐ Authorization is checked on every write/delete/admin route.
- ☐ Stripe webhook signature and event idempotency are tested.
- ☐ VIP downloads are not public file URLs.
- ☐ Input length, file type, upload size, and HTML are validated/sanitized.
- ☐ CORS permits only the production site origin.
- ☐ Rate limiting, bot protection, logs, and error monitoring are enabled.
- ☐ Database and uploaded media backups are automatic and restoration is tested.
- ☐ Privacy policy, terms, refund/cancellation policy, and contact information are published.
- ☐ Test mode and live mode each have the correct Price ID and webhook secret.

12. Troubleshooting

Site shows a directory or 404: Confirm index.html is at the hosting root and filename case matches links.

CSS/images missing: Check relative paths, capitalization, and whether an extra parent folder was uploaded.

Checkout button says not connected: Set window.SOS_BACKEND.apiUrl in js/backend-config.js and redeploy the front end.

Checkout returns 401: The server requires a verified identity. Local testing needs ALLOW_LOCAL_PREVIEW_IDENTITIES=true; production needs real auth middleware.

Payment succeeds but VIP stays locked: Inspect webhook delivery, signing secret, event subscriptions, server logs, payment_events, and member_profiles. Confirm client_reference_id/metadata userId exists.

Webhook signature fails: The webhook route must receive express.raw before express.json, and the endpoint must use the matching test/live signing secret.

Forums differ between browsers: They are still using localStorage. Complete the database/API migration.

Private music is downloadable by URL: Move it out of the public site and serve short-lived signed URLs after server-side entitlement checks.

Official references

- [GitHub Pages documentation](#)
- [GitHub Pages HTTPS](#)
- [Netlify deploy documentation](#)
- [Cloudflare Pages static HTML](#)
- [Vercel deployments](#)
- [Stripe Checkout Sessions](#)
- [Stripe fulfillment](#)
- [Stripe webhooks](#)
- [Supabase Auth](#)
- [Supabase Row Level Security](#)
- [Supabase Storage security](#)
- [cPanel Backup Wizard](#)

This guide is implementation guidance, not legal, tax, accounting, or payment-compliance advice. Provider interfaces, free-tier limits, and pricing can change; confirm current details in the linked official documentation before launch.
